

组成原理课程第 四 次实报告

实验名称：ALU 模块实现

学号： 2312331 姓名： 王一诺 班次： 张金老师

一、实验目的

- 1. 熟悉 MIPS 指令集中的运算指令，学会对这些指令进行归纳分类。
- 2. 了解 MIPS 指令结构。
- 3. 熟悉并掌握 ALU 的原理、功能和设计。
- 4. 进一步加强运用 verilog 语言进行电路设计的能力。
- 5. 为后续设计 cpu 的实验打下基础。

二、实验内容说明

针对组成原理第四次的 ALU 实验进行改进, 要求:

- 1、将原有的操作码进行位压缩, 调整操作码控制信号位宽为 4 位。
- 2、操作码调整成 4 位之后, 在原有 11 种运算的基础之上, 补充 3 种不同类型的运算 (要求一种大于置位比较, 一种位运算, 一种自选), 需要上实验箱或仿真验证计算结果。
- 3、注意改进实验上实验箱验证时, 操作码应该已经压缩到 4 位位宽。
- 4、实验报告中的原理图就用图 5.3 即可, 不再是顶层模块图。实验报告中讲清楚添加的三种运算过程, 还需要附上实验箱验证照片。
- 5、实验报告中要有介绍分析的内容, 针对实验箱照片, 要解释图中信息, 是否验证成功。

三、实验原理图

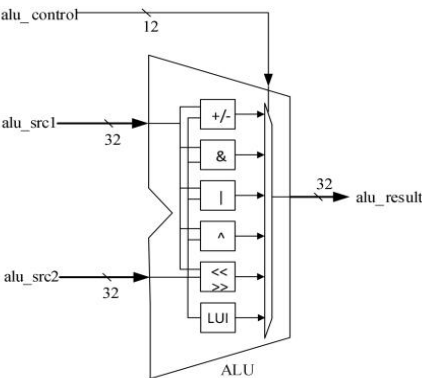


图 5.3 ALU 的原理图

原代码设计了 13 种运算（包括无操作），新增 3 种运算后总计 16 种运算，此时 ALU 操作可以统计为以下表：

序号	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
ALU 操作	无	加法	减法	有符号 比较小 于置位	无符号 比较小 于置位	按位 与	按位 或非	按位 或	按位 异或	逻辑 左移	逻辑 右移	算术 右移	高位 加载	有符号 比较大 于置位	按位 同或	按位 与非

四、实验步骤

1>位宽更改

首先修改 alu.v

我们对运算的控制信号位数进行了修改，由原先的 12 个二进制改为 4 位二进制控制信号，所以我们将位宽修改为[0:3]:

```
module alu(  
    input [3:0] alu_control, // ALU控制信号, 改为四位位宽  
    input [31:0] alu_src1, // ALU操作数1, 为补码  
    input [31:0] alu_src2, // ALU操作数2, 为补码  
    output [31:0] alu_result // ALU结果  
);
```

因为后面新增了 3 种运算，总计 16 种运算。编写控制信号选择操作的逻辑，如下图所示：

// 修改运算符选取逻辑，改为4位确定

```
assign alu_add = (~alu_control[3])&(~alu_control[2])&(~alu_control[1])&(alu_control[0]); //0001  
assign alu_sub = (~alu_control[3])&(~alu_control[2])&(alu_control[1])&(~alu_control[0]); //0010  
assign alu_slt = (~alu_control[3])&(~alu_control[2])&(alu_control[1])&(alu_control[0]); //0011  
assign alu_sltu = (~alu_control[3])&(alu_control[2])&(~alu_control[1])&(~alu_control[0]); //0100  
assign alu_and = (~alu_control[3])&(alu_control[2])&(~alu_control[1])&(alu_control[0]); //0101  
assign alu_nor = (~alu_control[3])&(alu_control[2])&(alu_control[1])&(~alu_control[0]); //0110  
assign alu_or = (~alu_control[3])&(alu_control[2])&(alu_control[1])&(alu_control[0]); //0111  
assign alu_xor = (alu_control[3])&(~alu_control[2])&(~alu_control[1])&(~alu_control[0]); //1000  
assign alu_sll = (alu_control[3])&(~alu_control[2])&(~alu_control[1])&(alu_control[0]); //1001  
assign alu_srl = (alu_control[3])&(~alu_control[2])&(alu_control[1])&(~alu_control[0]); //1010  
assign alu_sra = (alu_control[3])&(~alu_control[2])&(alu_control[1])&(alu_control[0]); //1011  
assign alu_lui = (alu_control[3])&(alu_control[2])&(~alu_control[1])&(~alu_control[0]); //1100  
// 新增三种运算——  
assign alu_sgt = (alu_control[3])&(alu_control[2])&(~alu_control[1])&(alu_control[0]); //1101  
assign alu_nxor = (alu_control[3])&(alu_control[2])&(alu_control[1])&(~alu_control[0]); //1110  
assign alu_nand = (alu_control[3])&(alu_control[2])&(alu_control[1])&(alu_control[0]); //1111
```

然后，我们还需要对应修改 alu_display.v:

调用模块时的 ALU 控制信号位宽需要更改:

```
//-----{调用ALU模块}begin  
reg [3:0] alu_control; // ALU控制信号, 改为四位位宽  
reg [31:0] alu_src1; // ALU操作数1  
reg [31:0] alu_src2; // ALU操作数2  
wire [31:0] alu_result; // ALU结果
```

同时，alu_control 的输入也要注意，输入的信号位宽也要更改:

```
//当input_sel为00时，表示输入数控制信号（改为四位位宽），即alu_control  
always @(posedge clk)  
begin  
    if (!resetn)  
    begin  
        alu_control <= 12'd0;  
    end  
    else if (input_valid && input_sel==2'b00)  
    begin  
        alu_control <= input_value[3:0];  
    end  
end  
end
```

2>新增三种运算

按照实验要求，在 alu.v 中新增了三个功能，先增加三个 wire 表示对应的选取信号，并写入控制信号选择逻辑中（见上文）。再增加三个 wire 存储对应运算结果。如下图：

```
// 新增三种运算
wire alu_sgt; //有符号比较, 大于置位, 复用加法器做减法
wire alu_nxor; //按位同或
wire alu_nand; //按位与非
```

```
// 新增三种运算
wire [31:0] sgt_result;
wire [31:0] nxor_result;
wire [31:0] nand_result;
```

然后, 编写新增的这三个功能模块。

①有符号比较, 大于置位: 与小于置位基本相同, 可以想到, 不满足小于置位且两个操作数不相等的情况就是大于置位, 于是用 `adder_result` 的或缩位运算结果 `adder_zero` 记录两操作数是否相等 (相等时, `adder_zero=0`, 此时不应置位), 那么大于置位结果就是小于置位取反后与上 `adder_zero`。如下图所示:

```
// sgt结果
// 不满足小于置位且两操作数不相等的就是大于置位
wire adder_zero;
assign adder_zero=|adder_result;
assign sgt_result[31:1] = 31'd0;
assign sgt_result[0] = ~( (alu_src1[31] & ~alu_src2[31]) | (~(alu_src1[31]^alu_src2[31]) & adder_result[31]) ) & adder_zero;
```

②按位同或: 按位异或结果取反就是按位同或的结果, 如下图所示:

```
assign xor_result = alu_src1 ^ alu_src2; // 异或结果为两数按位异或
assign nxor_result = ~xor_result; // 同或结果为异或取反
```

③按位与非: 实验要求最后自选一个功能实现, 我选择了按位与非。按位与结果逐位取反就是按位与非的结果, 如下图所示:

```
assign and_result = alu_src1 & alu_src2; // 与结果为两数按位与
assign nand_result = ~and_result; // 与非结果为与结果按位取反
```

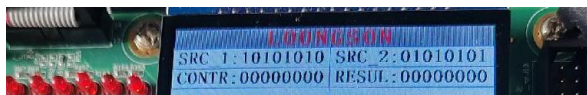
最后, 修改结果输出逻辑, 把新增的这三个运算结果加入输出逻辑中即可:

```
// 选择相应结果输出
assign alu_result = (alu_add|alu_sub) ? add_sub_result[31:0] :
    alu_slt ? slt_result :
    alu_sltu ? sltu_result :
    alu_and ? and_result :
    alu_nor ? nor_result :
    alu_or ? or_result :
    alu_xor ? xor_result :
    alu_sll ? sll_result :
    alu_srl ? srl_result :
    alu_sra ? sra_result :
    alu_lui ? lui_result :
    alu_sgt ? sgt_result : // 有符号数, 大于置位比较
    alu_nxor ? nxor_result : // 按位同或
    alu_nand ? nand_result : // 按位与非
    32'd0;

endmodule
```

五、实验结果分析

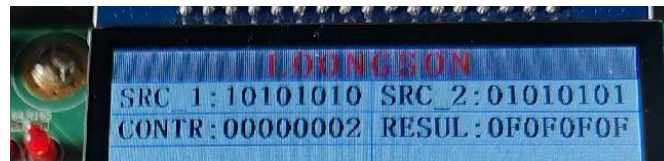
1. 复现



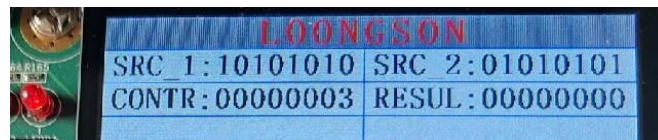
这里的 SRC_1 和 SRC_2 以及 RESULT 都是 16 进制数, 所以实际上的运算为:
 $00010000000100000001000000010000 \text{ (2)} + 00000001000000010000000100000001 \text{ (2)} =$
 $00010001000100010001000100010001 \text{ (2)} = 11111111 \text{ (16)},$ 验证正确, 如下图:



$00010000000100000001000000010000(2) - 00000001000000010000000100000001(2) = 00001111000011110000111100001111(2) = \text{FFFFFFF}(16)$ ，验证正确，如下图：



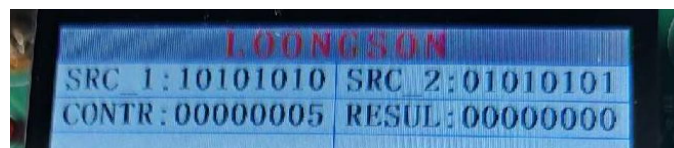
$00010000000100000001000000010000(2) < 00000001000000010000000100000001(2) = 0$ ，验证正确，如下图：



$00010000000100000001000000010000(2) < 00000001000000010000000100000001(2) = 0$ ，验证正确，如下图：



$00010000000100000001000000010000(2)$ 按位& $00000001000000010000000100000001(2) = 00000000000000000000000000000000(2) = 00000000(16)$ ，验证正确，如下图：



$00010000000100000001000000010000(2)$ 按位或非 $00000001000000010000000100000001(2) = 11101110111011101110111011101110(2) = \text{EEEEEEE}(16)$ ，验证正确，如下图：



$00010000000100000001000000010000(2)$ 按位或 $00000001000000010000000100000001(2) = 00010001000100010001000100010001(2) = 11111111(16)$ ，验证正确，如下图：



$00010000000100000001000000010000(2)$ 按位异或 $00000001000000010000000100000001(2) = 00010001000100010001000100010001(2) = 11111111(16)$ ，验证正确，如下图：



Src_1 的末 5 位为移位量， $00010000000100000001000000010000(2)$ 末五位也就是 10000，

根据移位逻辑，src_2 左移 16 位且空位置 0，得到 00000001000000010000000000000000 (2) = 01010000 (16)，验证正确，如下图：



与逻辑左移相似，也就是 src_2 右移 16 位且空位置 0，得到 000000000000000000001000100010001 (2) = 00000101 (16)，验证正确，如下图：



与逻辑右移相同，只是空位置符号位，这里 src_2 符号位为 0，所以结果仍是 00000101，验证正确，如下图：



高位加载是将 src_2 的低十六位放到寄存器的高十六位，结果的低十六位置 0，得到 00000001000000001000000000000000 (2) = 01010000 (16)，验证正确，如下图：

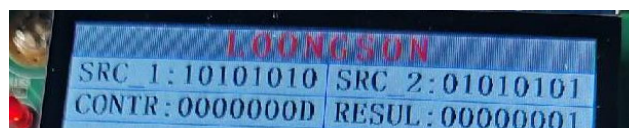


2. 新增三种运算

5.2.1 先验证有符号数比较，大于置位：

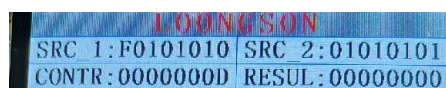
A>正数的比较：

000100000000100000001000000010000 (2) > 000000001000000010000000100000001 (2) = 1，验证正确，如下图所示：



B>负数与正数的比较：

111100000000100000001000000010000 (负数) > 000000001000000010000000100000001 (正数) = 0，验证正确，如下图所示：



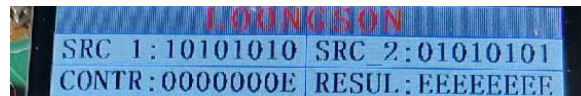
C>负数的比较：

11111111111111111111111111111011 (-5) > 11111111111111111111111111111010 (-6) = 1，验证正确，如下图所示：



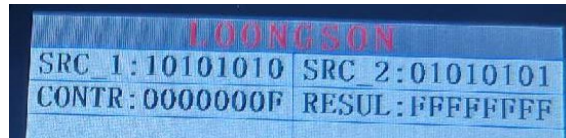
5.2.2 再验证按位同或:

00010000000100000001000000010000(2)按位同或 00000001000000010000000100000001(2)
= 11101110111011101110111011101110(2) = EEEEEEE(16), 验证正确, 如下图所示:



5.2.3 最后验证按位与非:

00010000000100000001000000010000(2)按位与非 00000001000000010000000100000001(2)
= 11111111111111111111111111111111(2) = FFFFFFFF(16), 验证正确, 如下图所示:



六、总结感想

本次实验没有太大难度, 但更像是前面所学内容的一次简单综合, 需要对 vivado 的熟练使用、对模块设计和 display 设计的深入理解、对设计流程以及上箱验证操作的整体性掌握, 这是对目前所学的整体小测试和巩固;

当然, 本次实验也让我具体深入地了解了 ALU 的设计和实现, 不同运算功能的设计和选控, 为后面 CPU 设计打下基础。